

**UNIVERSIDADE DO ESTADO DE SANTA CATARINA – UDESC  
CENTRO DE CIÊNCIAS TECNOLÓGICAS – CCT  
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO**

**KELWIN EFRAIN BAGNHUK DA SILVA,  
URIEL PACHECO DE SOUZA**

**SISTEMA DE GESTÃO DE FROTAS**

**Documento de Especificação – Fase 1**

**Joinville, SC  
2026**

# Sumário

|          |                                                           |          |
|----------|-----------------------------------------------------------|----------|
| <b>1</b> | <b>Descrição do Problema</b>                              | <b>2</b> |
| 1.1      | Escopo do Software . . . . .                              | 2        |
| 1.2      | Lista de Stakeholders . . . . .                           | 3        |
| <b>2</b> | <b>Requisitos do Software</b>                             | <b>3</b> |
| 2.1      | Requisitos Funcionais (RF) . . . . .                      | 3        |
| 2.2      | Requisitos Não-Funcionais (RNF) . . . . .                 | 4        |
| <b>3</b> | <b>Estimativa de Duração do Projeto Completo</b>          | <b>4</b> |
| 3.1      | Contagem de Elementos de Software . . . . .               | 5        |
| 3.2      | Pontos de Função Não Ajustados (PFNA) . . . . .           | 6        |
| 3.3      | Fator de Ajuste . . . . .                                 | 6        |
| 3.4      | Estimativa de Esforço e Duração (COCOMO Básico) . . . . . | 7        |
| 3.5      | Conclusão da Estimativa para a Equipe . . . . .           | 8        |
| <b>4</b> | <b>Diagrama de Classes do Projeto UML</b>                 | <b>8</b> |
| 4.1      | Padrões de Projeto Utilizados . . . . .                   | 8        |
| 4.2      | Imagem do Diagrama . . . . .                              | 9        |

# 1 Descrição do Problema

Atualmente, empresas que atuam no setor de logística e entregas dependem crucialmente da disponibilidade e eficiência de seus veículos. No entanto, essas organizações frequentemente realizam a gestão de suas frotas de maneira manual ou desorganizada, o que ocasiona um grande esforço desnecessário e conseqüentemente, uma lacuna de informações por falta de uma sistematização.

O problema central abordado por este projeto é a dificuldade dessas empresas em determinar com precisão o custo exato por quilômetro rodado de cada veículo da frota. Além disso, há uma grande dificuldade em acompanhar o desgaste e programar manutenções preventivas (como trocas de óleo, pneus e substituição de correias) de forma sistemática. Essa falta de controle resulta em um aumento nos gastos com combustível, alta incidência de manutenções corretivas (que são mais custosas e causam indisponibilidade imprevista do veículo) e, conseqüentemente, perda de rentabilidade operacional.

## 1.1 Escopo do Software

A solução será uma plataforma de gestão voltada para o controle de insumos e manutenções, projetada para fornecer métricas claras e automatizar o calendário de revisões da frota.

### No Escopo:

- **Cadastro e Gestão da Frota:** inserção de veículos com dados como placa, modelo, capacidade e quilometragem inicial.
- **Controle de Abastecimento:** registro de cada abastecimento contendo data, veículo, motorista, tipo de combustível, litros abastecidos, valor total e quilometragem no momento.
- **Gestão de Manutenção:** registro de manutenções realizadas (preventivas e corretivas) e custos de peças/serviços.
- **Geração de Alertas:** sistema de notificação automática para alertar o gestor sobre manutenções preventivas iminentes, baseando-se em quilometragem percorrida ou tempo.
- **Dashboard e Relatórios:** cálculo automático e visualização de indicadores-chave, como Custo por Km Rodado e Média de Consumo (Km/L) por veículo.

### Fora do Escopo:

- Rastreamento e telemetria de veículos em tempo real via hardware GPS.
- Roteirização automática ou otimização de rotas de entrega.
- Gestão financeira completa da empresa (como folha de pagamento ou emissão de notas fiscais).

## 1.2 Lista de Stakeholders

- **Gestor de Frota:** usuário principal da plataforma. Responsável por monitorar os indicadores, cadastrar manutenções e tomar decisões com base nos custos apresentados.
- **Motoristas:** usuários que interagem com o sistema (ou fornecem os dados) no dia a dia, registrando a quilometragem e os cupons fiscais no momento do abastecimento.
- **Proprietários da Empresa:** interessados no resultado final gerado pelo software, buscando a redução e otimização de custos.
- **Oficinas Parceiras:** partes afetadas indiretamente, que serão responsáveis por realizar as manutenções nos veículos indicadas pelo cronograma do sistema.

## 2 Requisitos do Software

### 2.1 Requisitos Funcionais (RF)

Os Requisitos Funcionais descrevem o que o sistema deve fazer, isto é, a descrição das funcionalidades.

**RF01 – Gerenciamento de Veículos:** o sistema deve permitir o cadastro, edição, visualização e inativação de veículos. Os dados mínimos são: placa, modelo, capacidade de carga e quilometragem atual (hodômetro).

**RF02 – Controle de Estado do Veículo:** o sistema deve controlar o status de cada veículo (Disponível, Em Operação, Em Manutenção).

**RF03 – Registro de Abastecimentos:** o sistema deve permitir o registro de abastecimentos, exigindo a inserção de dados como: veículo, data, motorista, tipo de combustível, litros abastecidos, valor pago e a quilometragem do veículo no momento do posto.

**RF04 – Registro de Manutenções:** o sistema deve permitir o cadastro de pedidos de manutenção (corretivas ou preventivas), registrando as peças trocadas, o custo do serviço e a quilometragem na qual a manutenção foi realizada.

**RF05 – Sistema de Alertas (Notificações):** o sistema deve emitir um alerta automático para o gestor quando a quilometragem atual do veículo atingir ou ultrapassar o limite estipulado para a próxima revisão (ex.: troca de óleo). (*Ideal para o padrão Observer*).

**RF06 – Cálculo de Indicadores (Dashboard):** o sistema deve ser capaz de calcular e exibir a média de consumo (Km/L) e o Custo por Quilômetro Rodado do veículo (CPK), utilizando a seguinte fórmula para o custo:

$$CPK = \frac{Custos\ Fixos + Custos\ Variveis}{Quilmetros\ Rodados} \quad (1)$$

**RF07 – Gerenciamento de Usuários (Gestores e Motoristas):** o sistema deve permitir o cadastro, edição e inativação de usuários, armazenando dados como nome, CNH (para motoristas) e credenciais de acesso.

**RF08 – Autenticação no Sistema (Login):** o sistema deve permitir que os usuários façam login de forma segura para utilizar a aplicação de acordo com os seus perfis (Gestor ou Motorista).

**RF09 – Cadastro de Oficinas Parceiras:** o sistema deve permitir o cadastro de oficinas e mecânicas (contendo dados como nome, CNPJ/CPF, telefone e endereço) para que possam ser relacionadas às manutenções.

## 2.2 Requisitos Não-Funcionais (RNF)

Os Requisitos Não-Funcionais descrevem as restrições tecnológicas e as regras de negócio associadas aos requisitos funcionais, uma vez que tais regras não descrevem uma nova funcionalidade do sistema, e sim uma restrição de comportamento sobre uma funcionalidade já existente.

**RNF01 – Linguagem e Paradigma:** o sistema deve ser desenvolvido utilizando uma linguagem compatível com o paradigma de Programação Orientada a Objetos (POO) e com os testes a serem realizados, visando a correta aplicação dos padrões de projeto e testes unitários. A linguagem utilizada neste projeto será Python.

**RNF02 – Testabilidade:** A estrutura do código deve focar em minimizar o acoplamento, permitindo que seja possível realizar os testes automatizados em cada classe individualmente e sem problemas através de um framework de testes (Nesse caso, *pytest*).

**RNF03 – Persistência de Dados (Simplificada):** para o escopo deste trabalho acadêmico, os dados poderão ser mantidos em memória durante a execução ou armazenados em arquivos de texto simples, sem a necessidade de um Banco de Dados Relacional complexo.

**RNF04 – Interface de Usuário:** o sistema interagirá com o usuário através de uma Interface de Linha de Comando, focando os esforços no desenvolvimento do backend e nas regras de negócio.

**RNF05 – Restrição de Transição de Estado:** o sistema não pode permitir que um veículo inicie uma operação se o seu estado atual for *Em Manutenção*. Esta restrição é resolvida estruturalmente através do padrão de projeto **State**.

**RNF06 – Atualização Automática de Hodômetro:** ao registrar um abastecimento, o sistema deve atualizar automaticamente a quilometragem atual (hodômetro) do veículo cadastrado.

**RNF07 – Controle de Acesso por Perfil:** o sistema deve diferenciar as permissões de acesso com base no perfil do usuário. Apenas o Gestor pode acessar o Dashboard, o Motorista pode apenas registrar abastecimentos.

## 3 Estimativa de Duração do Projeto Completo

Para estimar o esforço e a duração do desenvolvimento do Sistema de Gestão de Frotas, utilizou-se a técnica de Análise de Pontos de Função (APF) para mensurar o tamanho do software e o modelo COCOMO Básico para extrair a estimativa de cronograma.

### 3.1 Contagem de Elementos de Software

Com base no escopo e nos Requisitos Funcionais, foram identificadas as principais entidades que o sistema irá manipular:

| Descrição (Entidade – ALI) | Complexidade | PF        |
|----------------------------|--------------|-----------|
| Veículo (RF01, RF02)       | Média        | 10        |
| Abastecimento (RF03)       | Média        | 10        |
| Manutenção (RF04)          | Média        | 10        |
| Usuário (RF07, RF08)       | Média        | 10        |
| Oficina (RF09)             | Baixa        | 7         |
| <b>Total</b>               |              | <b>47</b> |

Tabela 1: Número de Arquivos Lógicos Internos

Avaliando as interações dos usuários com o sistema, estimamos as Entradas Externas (EE), Saídas Externas (SE) e Consultas Externas (CE):

#### Entradas Externas (EE)

- Cadastrar/Editar Veículo (Média) = 4 PF
- Cadastrar/Editar Usuário (Média) = 4 PF
- Cadastrar/Editar Oficina Parceira (Baixa) = 3 PF
- Registrar Abastecimento (Média) = 4 PF
- Registrar Manutenção (Média) = 4 PF
- Atualizar Status do Veículo (Baixa) = 3 PF
- Autenticação (Login) (Média) = 4 PF

**Subtotal (EE) = 26 PF**

#### Saídas Externas (SE)

- Dashboard de Indicadores (Custo/Km e Km/L) (Alta) = 7 PF
- Emissão de Alerta de Manutenção Preventiva (Média) = 5 PF

**Subtotal (SE) = 12 PF**

#### Consultas Externas (CE)

- Consulta de Frota/Veículos (Média) = 4 PF
- Consulta de Usuário/Motorista (Média) = 4 PF
- Consulta de Oficinas (Baixa) = 3 PF
- Consulta de Histórico de Abastecimentos/Manutenções (Média) = 4 PF

**Subtotal (CE) = 15 PF**

### 3.2 Pontos de Função Não Ajustados (PFNA)

O tamanho inicial do sistema dado pelos Pontos de Função Não Ajustados (PFNA), é calculado pela soma dos pontos de todos os elementos do software encontrados, conforme a fórmula geral da metodologia:

$$PFNA = \sum (\text{Elemento} \times \text{Peso})$$

Utilizando os subtotais já calculados para os Arquivos Lógicos Internos (ALI), Entradas Externas (EE), Saídas Externas (SE) e Consultas Externas (CE), temos a seguinte equação:

$$PFNA = \text{Subtotal (ALI)} + \text{Subtotal (EE)} + \text{Subtotal (SE)} + \text{Subtotal (CE)}$$

Substituindo pelos valores encontrados:

$$PFNA = 47 + 26 + 12 + 15 = 100 \text{ PF}$$

### 3.3 Fator de Ajuste

Avaliação das 14 questões de ajuste de complexidade para o cenário do projeto (notas de 0 a 5):

| N.º | Questão de Ajuste                                       | Nota | Justificativa                                                                                                       |
|-----|---------------------------------------------------------|------|---------------------------------------------------------------------------------------------------------------------|
| 1   | Necessita de backup?                                    | 3    | Os dados de abastecimento e manutenção são importantes, exigindo rotinas padrão de backup.                          |
| 2   | Necessita de mecanismos especializados de comunicação?  | 0    | O escopo excluiu explicitamente a telemetria via hardware GPS em tempo real.                                        |
| 3   | Tem processamento distribuído?                          | 0    | O processamento será centralizado em um único servidor/backend.                                                     |
| 4   | Precisa de alto desempenho?                             | 2    | Desempenho padrão exigido para aplicações web/mobile, sem processamento em tempo crítico.                           |
| 5   | Terá grande número de usuários em paralelo?             | 1    | Aplicação voltada para pequenas empresas (o gestor e um pequeno grupo de motoristas).                               |
| 6   | Precisará de entrada de dados on-line?                  | 5    | Essencial (nota máxima), pois motoristas e gestores irão introduzir dados de abastecimento e quilometragem on-line. |
| 7   | No caso de entradas on-line, existirão múltiplas telas? | 3    | Existirão telas distintas para gerir frotas, abastecimentos e visualização do dashboard.                            |
| 8   | A atualização das entidades será feita on-line?         | 3    | As atualizações de status (ex.: “Em Manutenção” para “Disponível”) ocorrem diretamente no sistema on-line.          |

| N.º                                 | Questão de Ajuste                                            | Nota      | Justificativa                                                                                                 |
|-------------------------------------|--------------------------------------------------------------|-----------|---------------------------------------------------------------------------------------------------------------|
| 9                                   | As entradas e saídas de dados serão complexas?               | 2         | Os formulários (placa, km, custo) e o dashboard são relativamente diretos e simples.                          |
| 10                                  | O processamento interno será complexo?                       | 3         | Nível intermediário, devido ao cálculo automático de indicadores contínuos (Custo/Km e Km/L).                 |
| 11                                  | O código será projetado para ser reutilizado?                | 2         | A arquitetura será modular (POO) para permitir testes unitários isolados, facilitando a reutilização interna. |
| 12                                  | Migração e instalação estarão incluídos?                     | 0         | Por se tratar de um protótipo acadêmico inicial, não há migração de dados legados.                            |
| 13                                  | O sistema será instalado em diversas organizações?           | 1         | Desenhado para pequenas transportadoras em geral, mas com instalação focada inicialmente em nuvem própria.    |
| 14                                  | O projeto pretende facilitar mudanças e operação do usuário? | 3         | É importante ter uma interface amigável (CLI/GUI) para os usuários inserirem os dados diários com facilidade. |
| <b>Soma Total (Fator de Ajuste)</b> |                                                              | <b>28</b> |                                                                                                               |

Tabela 2: Definição das Questões de Ajuste

Cálculo dos Pontos de Função Ajustados (PF):

$$PF = PFNA \times (0,65 + 0,01 \times \text{Soma das Respostas}) \quad (2)$$

$$PF = 100 \times (0,65 + 0,01 \times 28) \quad (3)$$

$$PF = 100 \times 0,93 = 93 \text{ PF} \quad (4)$$

O sistema será desenvolvido na linguagem Python. Como o Python não constava na tabela de referência oferecida nos slides da disciplina (baseada em métricas mais antigas), buscamos uma atualização mais recente das métricas de Capers Jones (2017), que define 53,33 LOC/PF para a linguagem.

$$LOC = 93 \times 53,33 \approx 4960 \text{ linhas de código} \quad (5)$$

$$KLOC = 4,96 \text{ (milhares de linhas)} \quad (6)$$

### 3.4 Estimativa de Esforço e Duração (COCOMO Básico)

O projeto enquadra-se no nível “Básico” (projetos simples, com equipe pequena) do modelo paramétrico de estimativas. As fórmulas do COCOMO do Nível Básico são:

$$\text{Esforço} = 2,4 \times (KLOC)^{1,05} \quad (7)$$

$$\text{Duração} = 2,5 \times (\text{Esforço})^{0,38} \quad (8)$$



### Calculando o Esforço:

$$\text{Esforço} = 2,4 \times (4,96)^{1,05} \quad (9)$$

$$\text{Esforço} \approx 2,4 \times 5,372 \approx 12,89 \text{ pessoas-mês} \quad (10)$$

### Calculando a Duração Cronológica:

$$\text{Duração} = 2,5 \times (12,89)^{0,38} \quad (11)$$

$$\text{Duração} \approx 2,5 \times 2,642 \approx 6,605 \text{ meses} \quad (12)$$

## 3.5 Conclusão da Estimativa para a Equipe

O modelo COCOMO Básico estima um esforço total de desenvolvimento de aproximadamente **12,89 pessoas-mês**, o que representa a quantidade total de trabalho necessária, independentemente de quantas pessoas a realizam. Considerando uma equipe de 2 integrantes dividindo o esforço de forma equilibrada, cada um teria uma carga de trabalho de aproximadamente  $12,89 \div 2 \approx 6,445$  pessoas-mês de trabalho intensivo. No entanto, a fórmula de Duração do COCOMO indica que o tempo cronológico ideal para percorrer todas as fases do projeto (planejamento, construção e testes) sem sobrecarga ou erros estruturais é de aproximadamente **6,605 meses**, que arredondando são **7 meses**.

## 4 Diagrama de Classes do Projeto UML

O modelo de classes foi construído com base nos Requisitos Funcionais e Não-Funcionais. Para resolver problemas específicos de mudança de comportamento e emissão de alertas, foram aplicados os padrões de projeto **State** e **Observer**.

### 4.1 Padrões de Projeto Utilizados

1. **State (Estado):** aplicado no controle do status do veículo (RF02, RNF05). Como um veículo não pode iniciar uma operação se estiver em manutenção, o padrão State encapsula os comportamentos permitidos para cada estado atual do veículo (Disponível, Em Operação e Em Manutenção).
2. **Observer (Observador):** aplicado no sistema de alertas de manutenção (RF05). A classe **Veiculos** implementa a interface **Subject** (sujeito observável). Sempre que a quilometragem é atualizada (por um abastecimento), ela notifica o **Gestor** – que implementa a interface **Observer** – caso o limite para a próxima revisão seja atingido.

## 4.2 Imagem do Diagrama

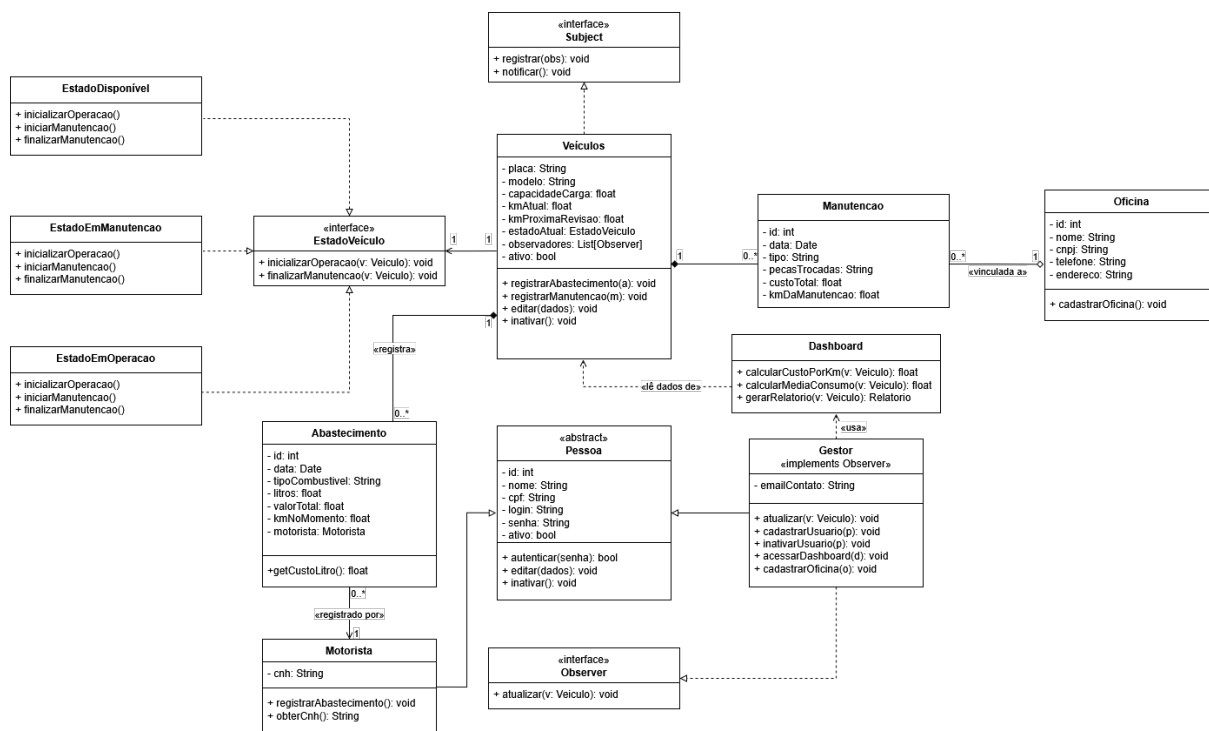


Figura 1: Diagrama de Classes UML

## Referências

JONES, Capers. **Software Economics and Function Point Metrics: Thirty years of IFPUG Progress.** Version 10.0. [S. l.]: Namcook Analytics LLC, 2017.

MAXIFROTA. **Saiba como calcular o CPK para sua frota.** [S. l.]: Maxifrota, 2024. Disponível em: <https://maxifrota.com.br/blog/cpk/>. Acesso em: 20 jun. 2026.

VALENTE, Marco Tulio. **Engenharia de Software Moderna: Princípios e Práticas para Desenvolvimento de Software com Produtividade.** [S. l.]: Independente, 2020. Disponível em: <https://engsoftmoderna.info/>. Acesso em: 20 jun. 2026.